

Department of Electrical Engineering
University of Moratuwa



EE2044 – Electrical Measurements and Instrumentation

Automatic Cricket Ground Control
System

230511A - Rajapaksha J.S.W.

230516T - Rajapakshe R.P.M.T.

230517X - Rajapakshe W.R.K.L.M.H.N.

Table of Contents

1 PROJECT OVERVIEW	3
2 HARDWARE IMPLEMENTATION	4
2.1 Light Intensity Measurement System	4
2.2 Temperature and Humidity Monitoring	5
2.3 Soil Moisture Monitoring	5
2.4 Rain Detection System	6
2.5 Actuator System	6
3 SOFTWARE IMPLEMENTATION	6
3.1 LabVIEW based Control System	6
3.2 Arduino-Based Actuator Control	9
3.3 Firebase Realtime Database	13
4 WEB-BASED MONITORING PLATFORM	14
5 MACHINE LEARNING BASED PITCH PREDICTION	15
6 CONCLUSION	16
7 REFERENCES	17

List of Figures

Figure 1: Transimpedance Amplifier Circuit	4
Figure 2: Front Panel of the LabVIEW	7
Figure 3: Lighting Control Logic of the LabVIEW	7
Figure 4: Sensor Inputs and Actuator Control Logic in LabVIEW	8
Figure 5: Firebase Data Update Control in LabVIEW	9
Figure 6: Website Dashboard	14

List of Tables

Table 1: Light Intensity vs. Voltage Output	Error! Bookmark not defined.
---	------------------------------

PROJECT OVERVIEW

The Automated Cricket Ground Control System is an intelligent monitoring and control platform designed to improve the management of cricket ground conditions through automation, real-time monitoring, cloud connectivity, and machine learning. The system continuously collects environmental data using multiple sensors installed on the cricket ground and automatically controls ground equipment based on predefined conditions.

The primary objective of the system is to reduce manual intervention in cricket ground maintenance while providing accurate and timely information regarding ground conditions. The collected sensor data is processed using National Instruments (NI) LabVIEW software and stored in Firebase Realtime Database, enabling remote monitoring through a web-based dashboard.

The system utilizes a custom-designed light intensity measurement circuit, temperature and humidity monitoring, soil moisture sensing, and rain detection to evaluate the environmental conditions of the cricket ground. Based on these measurements, the system automatically controls irrigation pumps, linear actuators, and alarm systems to maintain optimal ground conditions.

In addition to automation features, the system incorporates a Machine Learning-based pitch prediction module. The prediction model utilizes both sensor data and external weather information obtained through a weather API to generate detailed pitch reports. These reports provide insights regarding pitch behavior, including pitch type, bounce characteristics, spin assistance, and seam movement, which can be valuable for players, coaches, and ground staff.

The complete system consists of four major subsystems:

1. Sensor Data Acquisition System
2. Automated Control System
3. Cloud Database and Web Monitoring System
4. Machine Learning-Based Pitch Prediction System

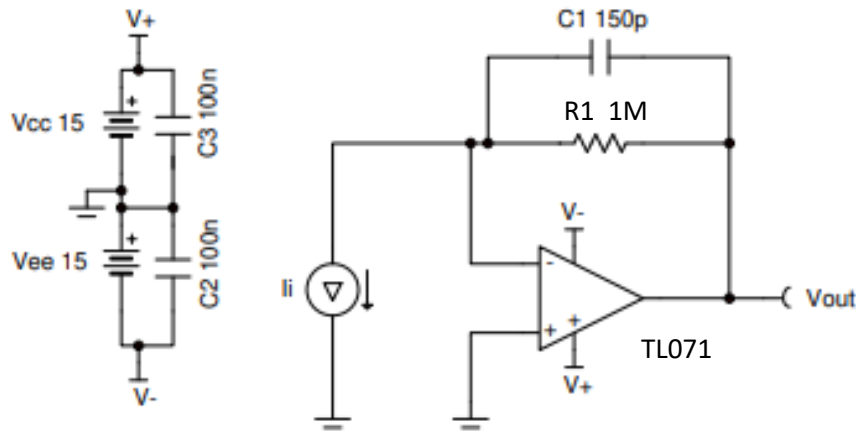
The integration of these subsystems creates a comprehensive smart cricket ground management platform capable of monitoring environmental conditions, controlling ground equipment, and providing intelligent predictions in real time.

HARDWARE IMPLEMENTATION

The hardware section of the system consists of multiple sensors and actuators connected to the data acquisition and control units.

2.1 Light Intensity Measurement System

Unlike conventional systems that use Light Dependent Resistors (LDRs), this project implements a photodiode-based light intensity measurement system to achieve higher accuracy and faster response times. The photodiode generates a current proportional to the incident light intensity. Since the generated current is very small, a Transimpedance Amplifier (TIA) circuit was designed using a TL071 operational amplifier to convert the photocurrent into a measurable voltage signal.



[1]

Figure 1: Transimpedance Amplifier Circuit

The output voltage of the transimpedance amplifier is directly proportional to the light intensity and can be calibrated to estimate the illumination level in lux. This approach provides better linearity and stability compared to traditional LDR-based circuits.

We calibrated

$$Lux = 163.16 \times V_{out} - 22.29$$

linear relationship with

0.9998 confidence with photo diode. Measured data table is given below.

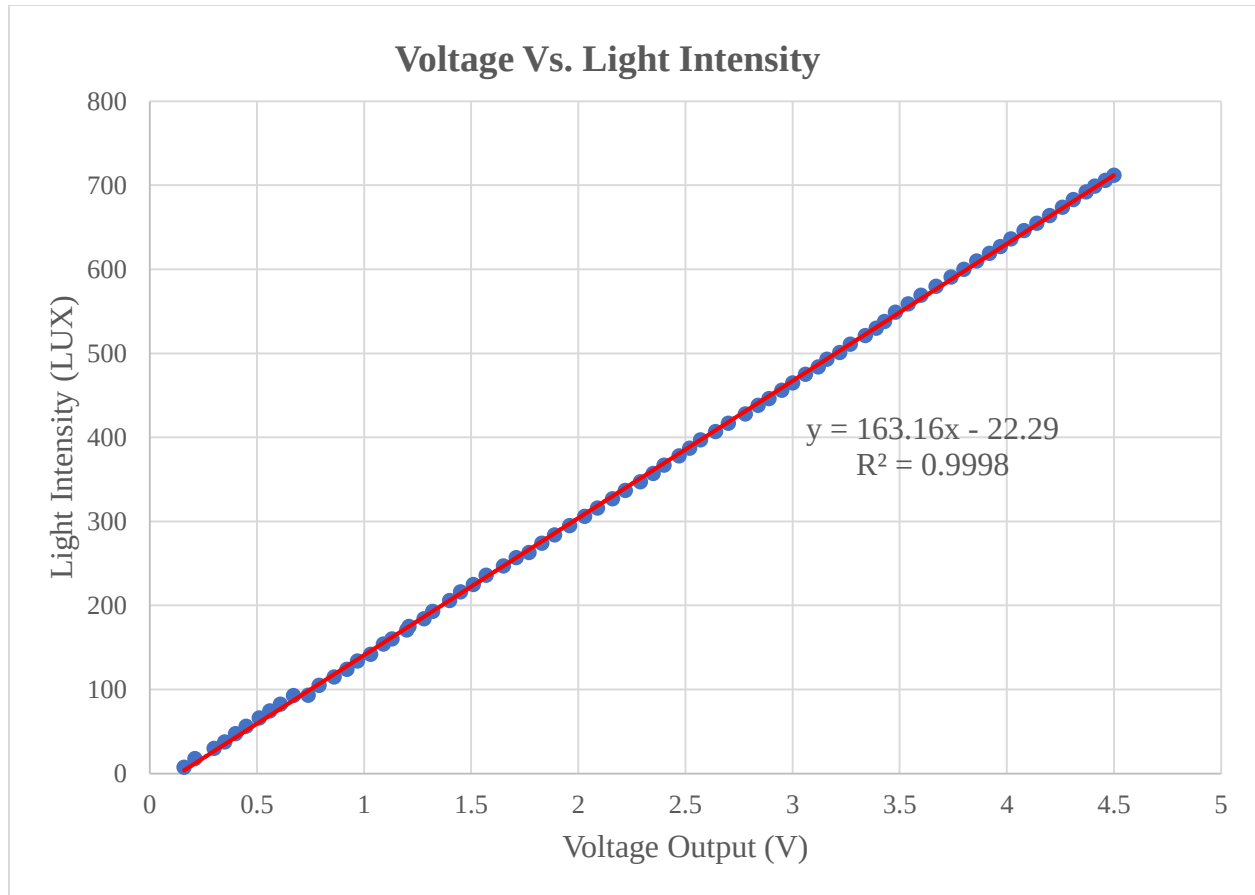


Figure 2: Light Intensity vs. Voltage Output

2.2 Temperature and Humidity Monitoring

A DHT11 sensor is used to measure ambient temperature and relative humidity around the cricket ground. These parameters significantly influence pitch conditions and overall playing environment. The measured values are continuously monitored and transmitted to the control system for further processing.

2.3 Soil Moisture Monitoring

The system utilizes a capacitive soil moisture sensor to determine the moisture content of the ground. Capacitive sensors are preferred over resistive sensors due to their higher durability, resistance to corrosion, and improved measurement stability. The measured soil moisture values are used to determine irrigation requirements and support pitch condition analysis.

2.4 Rain Detection System

A rain sensor module is incorporated to detect rainfall events. When rain is detected, the system generates alerts and initiates protective actions through the actuator subsystem. Rain detection also contributes to pitch condition assessment and irrigation control decisions.

2.5 Actuator System

The actuator subsystem consists of two linear actuators, two water pumps, and a buzzer module.

The linear actuators are responsible for mechanical operations such as opening and closing underground irrigation system. The water pumps are used to automate irrigation processes when the soil moisture level falls below a predefined threshold. The buzzer module provides audible alerts whenever important events such as rainfall detection or abnormal conditions occur.

SOFTWARE IMPLEMENTATION

The software architecture of the project is developed using NI LabVIEW, Arduino Uno, Firebase Realtime Database, and a web-based dashboard.

3.1 LabVIEW based Control System

NI LabVIEW serves as the central processing and control platform of the project. Sensor readings obtained through the data acquisition system are processed within LabVIEW, where control decisions are made according to predefined operating conditions.

The software continuously monitors environmental parameters and automatically controls actuators based on the collected data. For example, when the soil moisture level falls below a specified threshold, the irrigation pumps are activated. Similarly, when rainfall is detected, the system can trigger alarms and execute protective actions.

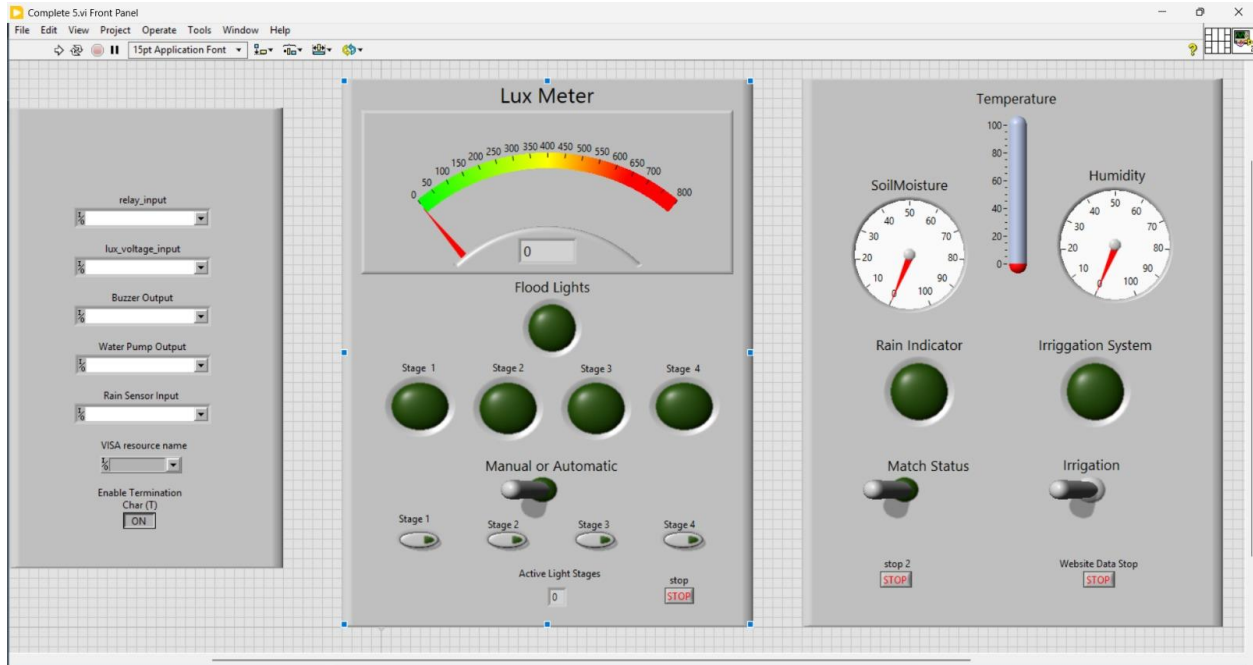


Figure 3: Front Panel of the LabVIEW

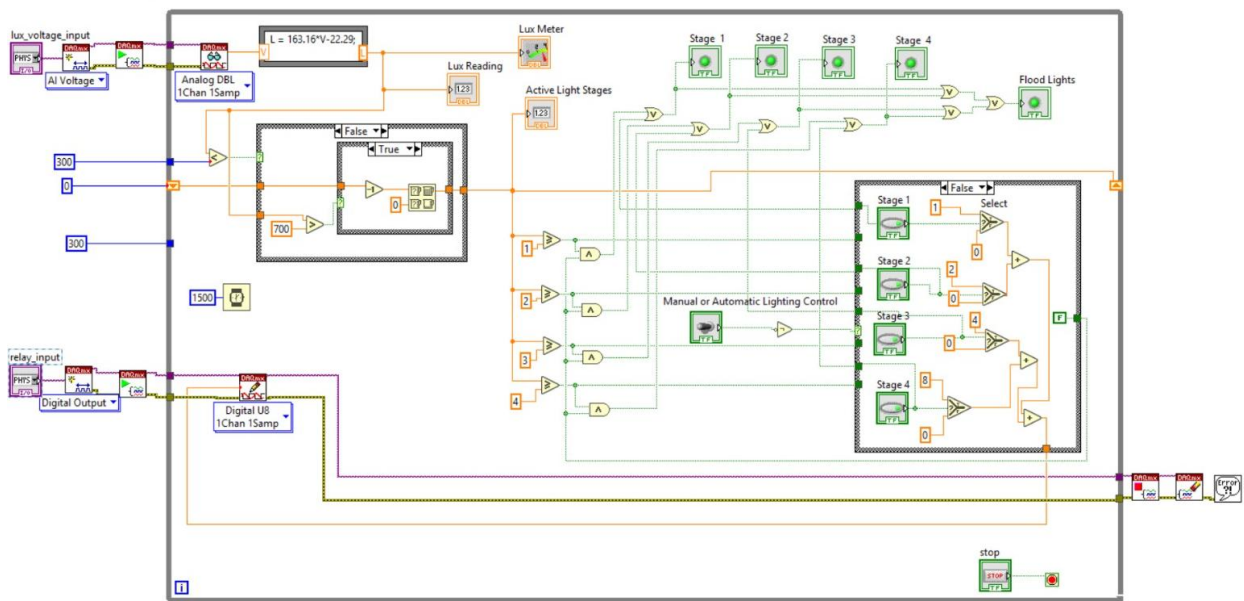


Figure 4: Lighting Control Logic of the LabVIEW

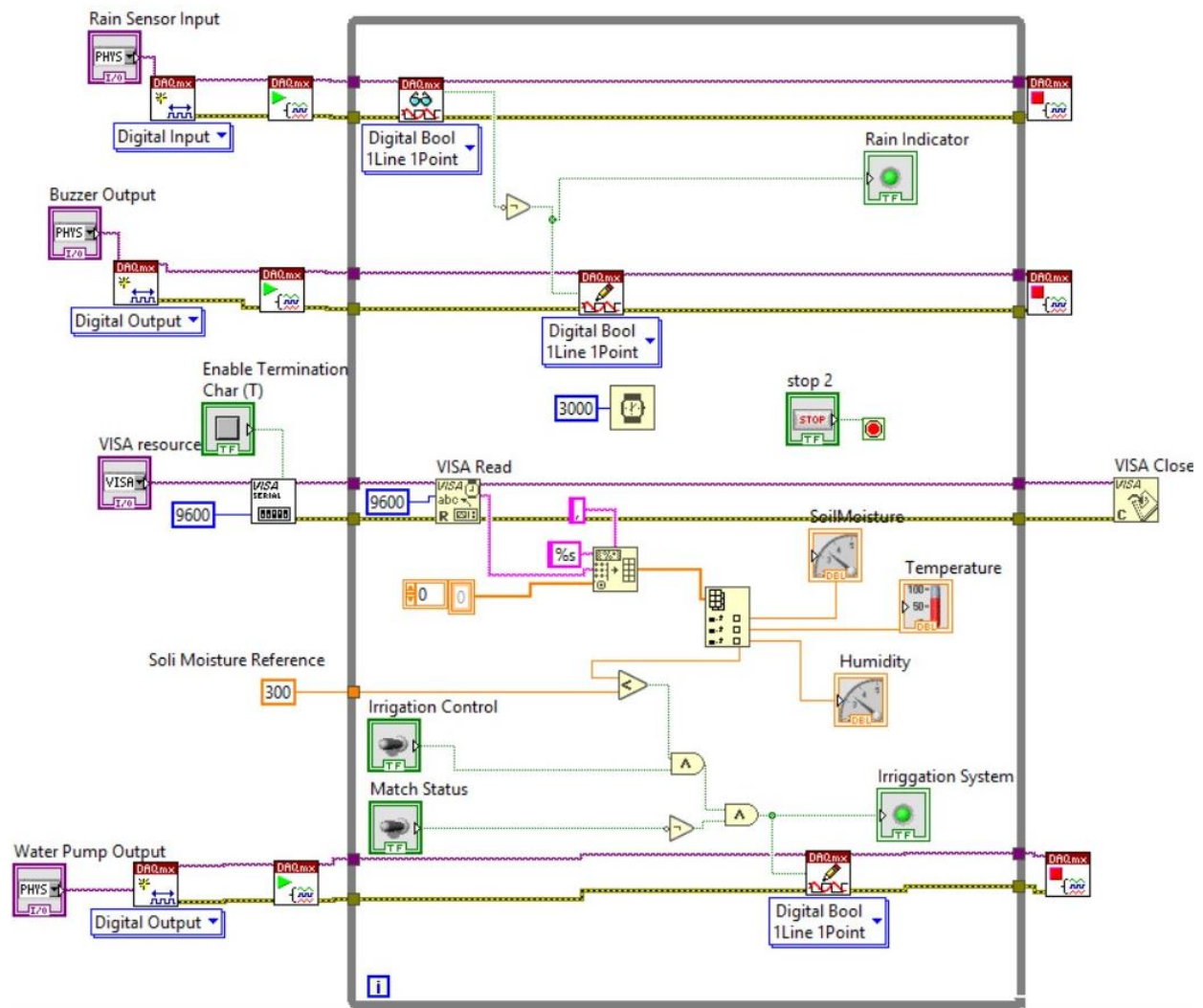


Figure 5: Sensor Inputs and Actuator Control Logic in LabVIEW


```

// ----- DHT11 -----
#define DHTPIN 11
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// ----- SERVO -----
Servo servo1;

// ----- PINS -----
const int daqPin = 2; // DAQ input
const int motorOut = 5; // motor output

const int moisturePin1 = A0;
const int moisturePin2 = A1;

const int servoPin1 = 6;

const int buzzerDAQ = 10; // DAQ input to the buzzer
const int buzzerOut = 9; // Buzzer output signal

// ----- MOISTURE CALIBRATION -----
const int DRY_VALUE = 600;
const int WET_VALUE = 0;

// ----- SERVO POSITIONS -----
const int LOW_POS = 0;
const int HIGH_POS = 90;

int currentPos = LOW_POS;

// ----- TIMING -----
unsigned long lastReadTime = 0;
const unsigned long readInterval = 3000;

// ----- SETUP -----
void setup()
{
    pinMode(daqPin, INPUT);
    pinMode(motorOut, OUTPUT);

    digitalWrite(motorOut, LOW);

    pinMode(moisturePin1, INPUT);
    pinMode(moisturePin2, INPUT);

```

```

    pinMode(buzzerDAQ, INPUT);
    pinMode(buzzerOut, OUTPUT);

    servo1.attach(servoPin1);

    Serial.begin(9600);

    dht.begin();

    servo1.write(LOW_POS);
    currentPos = LOW_POS;
}

// ----- LOOP -----
void loop()
{
    // ----- Buzzer -----
    if (digitalRead(buzzerDAQ) == HIGH){
        digitalWrite(buzzerOut, HIGH);
    }
    if (digitalRead(buzzerDAQ) == LOW) {
        digitalWrite(buzzerOut, LOW);
    }

    // ----- SENSOR READINGS -----
    if (millis() - lastReadTime >= readInterval)
    {
        lastReadTime = millis();

        int raw1 = analogRead(moisturePin1);
        int raw2 = analogRead(moisturePin2);

        int percent1 = map(raw1, DRY_VALUE, WET_VALUE, 0, 100);
        int percent2 = map(raw2, DRY_VALUE, WET_VALUE, 0, 100);

        percent1 = constrain(percent1, 0, 100);
        percent2 = constrain(percent2, 0, 100);

        float avgMoisture = (percent1 + percent2) / 2;

        float temp = dht.readTemperature();
        float hum = dht.readHumidity();

        if (isnan(temp) || isnan(hum))
        {

```

```

        temp = 0.0;
        hum = 0.0;
    }

    Serial.print(avgMoisture, 1);
    Serial.print(",");
    Serial.print(temp, 1);
    Serial.print(",");
    Serial.println(hum, 1);
}

// ----- CONTROL LOGIC -----
int state = digitalRead(daqPin);

// HIGH STATE → MOVE UP → AFTER 1s SET PIN 12 HIGH
if (state == HIGH && currentPos != HIGH_POS)
{
    //Serial.println("Moving to HIGH position");

    moveSmooth(currentPos, HIGH_POS);
    currentPos = HIGH_POS;

    delay(1000);
    digitalWrite(motorOut, HIGH);
    //Serial.println("Pin 12 = HIGH");
}

// LOW STATE → PIN 12 LOW IMMEDIATELY → WAIT → MOVE DOWN
else if (state == LOW && currentPos != LOW_POS)
{
    digitalWrite(motorOut, LOW);
    //Serial.println("Pin 12 = LOW");

    delay(1000);

    //Serial.println("Moving to LOW position");

    moveSmooth(currentPos, LOW_POS);
    currentPos = LOW_POS;

    digitalWrite(motorOut, LOW);
}

// Safety: if servo is LOW position, output must stay LOW

```

```

    if (currentPos == LOW_POS)
    {
        digitalWrite(motorOut, LOW);
    }

    delay(50);
}

// ----- SERVO SMOOTH MOVE -----
void moveSmooth(int fromPos, int toPos)
{
    if (fromPos < toPos)
    {
        for (int i = fromPos; i <= toPos; i++)
        {
            servo1.write(i);
            delay(20);
        }
    }
    else
    {
        for (int i = fromPos; i >= toPos; i--)
        {
            servo1.write(i);
            delay(20);
        }
    }
}

```

3.3 Firebase Realtime Database

Firebase Realtime Database is used as the cloud storage platform for the project. All sensor readings, actuator statuses, weather information, and machine learning predictions are stored in the database.

The use of Firebase enables real-time synchronization between the control system and the web application, ensuring that users can access the latest information from any location with internet connectivity.

WEB-BASED MONITORING PLATFORM

A modern web application was developed to provide remote access to the system. The website displays real-time sensor values, actuator statuses, weather information, and pitch prediction results through an interactive dashboard.

The web application was developed using React and TypeScript and is hosted using Firebase Hosting. The platform allows users to monitor cricket ground conditions without requiring direct access to the control hardware.

Website - <https://cricket-ground-control.web.app/>

The dashboard provides live updates of:

- Light intensity
- Temperature
- Humidity
- Soil moisture
- Rain status
- Actuator status
- Weather conditions
- Pitch prediction results

This cloud-based monitoring approach significantly improves accessibility and operational convenience.

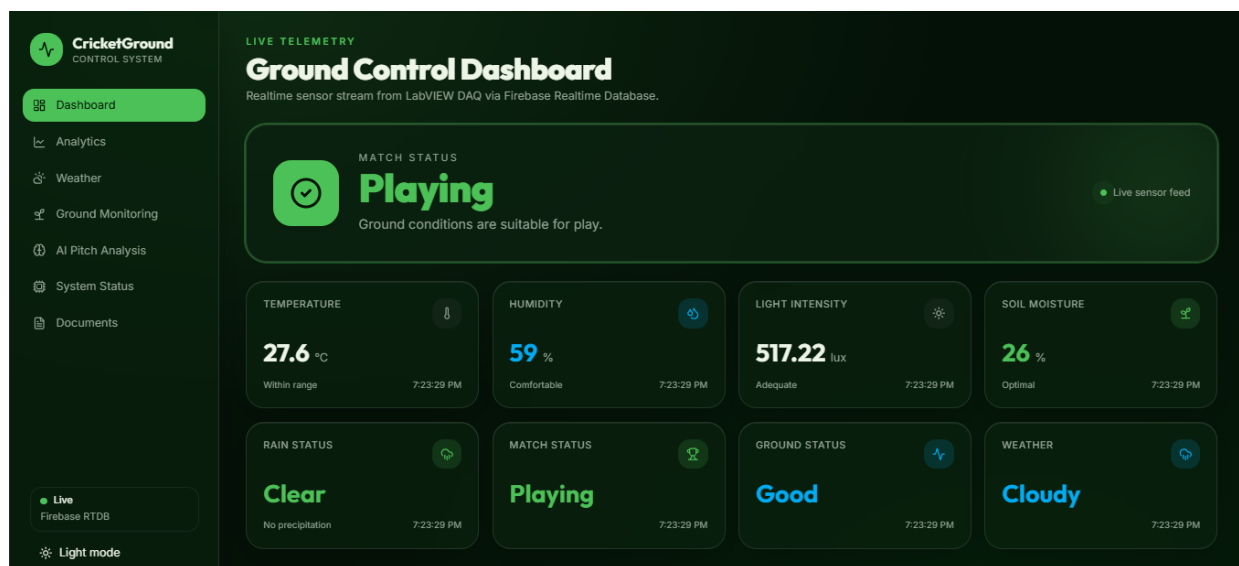


Figure 7: Website Dashboard

MACHINE LEARNING BASED PITCH PREDICTION

One of the key innovations of the project is the implementation of a Machine Learning-based pitch prediction system. The prediction model utilizes both locally measured sensor data and external weather data to generate a comprehensive pitch report.

The sensor inputs used by the model include:

- Light intensity
- Temperature
- Humidity
- Soil moisture
- Rain status

In addition to sensor measurements, weather information is obtained from an external WeatherAPI.

The weather parameters include:

- Dew point
- Cloud percentage
- Wind speed

These features are processed by the Machine Learning model to generate predictions regarding pitch characteristics.

The model predicts:

- Pitch Type (Batting Friendly, Balanced, Bowling Friendly)
- Bounce Characteristics (High & Consistent, Medium, Low)
- Spin Assistance (High, Moderate, Low)
- Seam Movement (High, Moderate, Low)

The generated pitch report is stored in Firebase Realtime Database and displayed through the web application, providing users with valuable insights regarding expected playing conditions.

Model - <https://github.com/janindusathsara/Automatic-Cricket-Ground-Control-backend>

CONCLUSION

The Automated Cricket Ground Control System successfully demonstrates the integration of sensor technology, industrial control software, cloud computing, web development, and machine learning within a single platform. The system provides real-time monitoring, automated control, cloud-based data management, and intelligent pitch prediction capabilities.

The implementation of accurate environmental sensing, automated irrigation control, remote monitoring, and machine learning-based analysis contributes to improved cricket ground management and reduced manual intervention. The developed system serves as a practical example of how modern engineering technologies can be applied to sports infrastructure management and smart automation systems.

REFERENCES

- [1] Texas Instruments, "Texas Instruments," 1 February 2018. [Online]. Available: https://www.ti.com/lit/an/sboa268b/sboa268b.pdf?ts=1782146852562&ref_url=https%253A%252F%252Fwww.ti.com [Accessed 20 June 2026].